CardioIA — Fase 3

Relatório técnico — Parte 2: Comunicação MQTT, Fog/Cloud e dashboard de monitoramento

> ⚠️ **AVISO CLÍNICO:** Este sistema é um protótipo acadêmico e **NÃO** substitui avaliação médica. Supervisionado por Dra. Fernanda Fassina (CRM-SP 169944).

Equipe (Grupo S — FIAP): Caroline de Castro Corrêa (RM567255), Enzo França Sader (RM566928), Lucas Hideki Oliveira Koyama (RM566925), Rodrigo Dias Figueiroa (RM567800) e Tiago Lindgren Curi (RM567016).

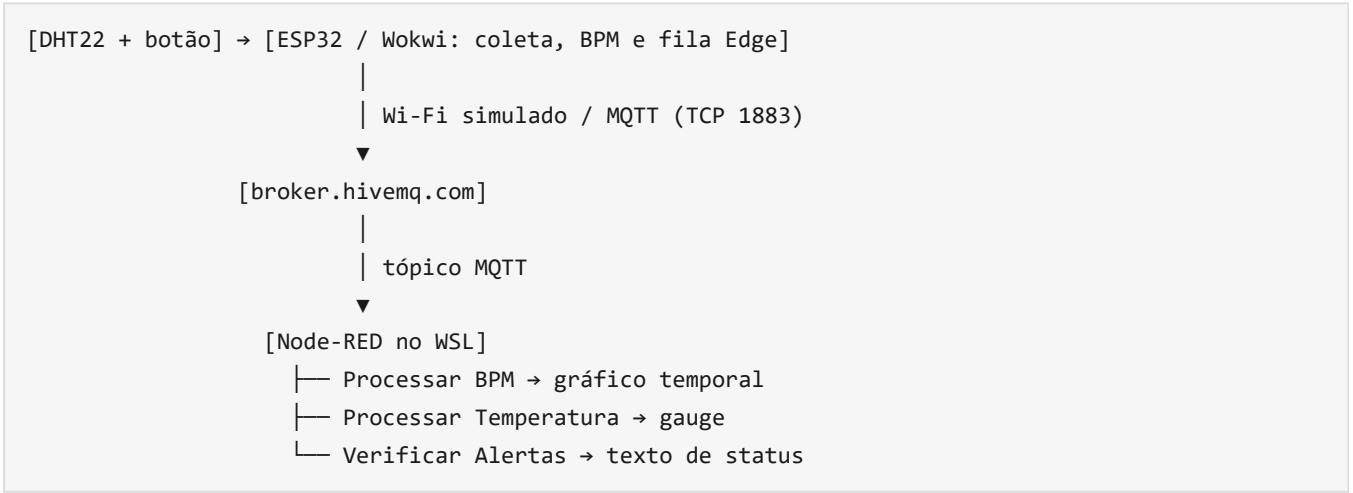
1. Objetivo e escopo

A segunda parte da Fase 3 do CardioIA demonstra a transmissão de dados de um protótipo de monitoramento baseado em IoT para uma camada de processamento e visualização. O dispositivo é um ESP32 **simulado no Wokwi**, conectado a um sensor DHT22 e a um botão de pressão que representa pulsos cardíacos manuais. O objetivo não é realizar diagnóstico médico, mas implementar e demonstrar coleta periódica, comunicação por MQTT, recepção de eventos e alertas em um dashboard.

A solução aproveita a etapa anterior de Edge Computing: as amostras são produzidas e armazenadas localmente em uma fila circular com capacidade máxima de 1.800 registros. Quando a publicação está indisponível, novas leituras continuam sendo coletadas; ao restabelecer a transmissão, o ESP32 tenta enviar os registros pendentes na ordem de chegada.

2. Arquitetura e componentes

O fluxo de informações segue esta arquitetura:



Camada Edge: o ESP32 lê temperatura e umidade do DHT22 a cada dois segundos, interpreta os acionamentos do botão como pulsos simulados e guarda os dados localmente. O BPM é estimado pelo intervalo entre pulsos; sem intervalo recente válido, a leitura é marcada como inválida. O uso de `bpm_valido` impede que o valor zero seja confundido com uma medição clínica de frequência cardíaca.

Mensageria: a biblioteca `PubSubClient` publica mensagens JSON em um broker MQTT de testes. O broker usado é `broker.hivemq.com`, porta `1883`, sem TLS e sem autenticação. O tópico configurado é `cardioia/fiap/fase3/enzf-47736293720720641/sinais`.

Camada de visualização (Fog/aplicação): o Node-RED é executado localmente no ambiente WSL e atua como cliente MQTT inscrito no mesmo tópico. As mensagens alimentam um gráfico de BPM, um medidor de temperatura ambiental e um indicador textual de alertas. A arquitetura demonstra integração com um broker remoto, mas **não equivale à implantação de uma infraestrutura própria de nuvem ou a um serviço clínico de produção**.

3. Estrutura das mensagens

Cada amostra publicada possui um identificador sequencial, um instante relativo à inicialização do ESP32 (`millis()`), os dados do sensor e a indicação de validade do BPM. Exemplo representativo:

```
{
  "dispositivo": "cardioia-esp32-01",
  "id": 9,
  "timestamp_ms": 18000,
  "temperatura": 24.0,
  "umidade": 40.0,
  "bpm": 0.0,
  "bpm_valido": false
}
```

`timestamp_ms` **não é um horário civil absoluto**: representa o tempo decorrido desde o início da simulação. Os campos `temperatura` e `umidade` são leituras ambientais do DHT22; portanto, um limiar de temperatura neste protótipo **não identifica febre**. O BPM é produzido por acionamentos manuais do botão e não por um sensor fisiológico real.

4. Resiliência e reenvio de dados

O firmware possui uma fila circular de 1.800 leituras. Com coletas a cada dois segundos, essa capacidade corresponde, em condições nominais, a aproximadamente uma hora de registros. Quando a fila fica cheia, o item mais antigo é substituído e o sistema registra a quantidade de descartes. Esse armazenamento é **volátil, em memória RAM**: reiniciar ou desligar o ESP32 elimina as amostras ainda não enviadas.

No Monitor Serial, o comando `f` desabilita a publicação e permite demonstrar a retenção de leituras; `o` habilita novamente a transmissão; e `s` apresenta o status. Após a reconexão ao Wi-Fi e ao broker, o firmware tenta publicar os dados acumulados em lotes limitados por passagem do loop, evitando concentrar todo o processamento em uma única operação.

A confirmação de `mqtt.publish(...)` indica que a biblioteca aceitou a publicação naquela conexão. **Não demonstra, isoladamente, entrega persistente no broker ou consumo pelo Node-RED**. A implementação utiliza publicação MQTT com QoS 0; portanto, não oferece confirmação fim a fim nem garantia de entrega. Para uma aplicação real, seriam necessários níveis apropriados de QoS, persistência, tratamento de duplicações e uma estratégia explícita de confirmação dos dados recebidos.

5. Implementação do dashboard no Node-RED

O fluxo foi construído usando o Node-RED e os widgets da biblioteca `@flowfuse/node-red-dashboard` (Dashboard 2.0). Um nó `mqtt in` recebe os eventos do tópico do CardioIA e os encaminha a três funções independentes:

1. **Processar BPM:** verifica se `bpm_valido` é verdadeiro e se `bpm` é numérico. Somente as leituras válidas seguem ao gráfico de linhas, cuja escala representa BPM.
2. **Processar Temperatura:** verifica a presença de valor numérico em `temperatura` e o envia ao widget de medidor (*gauge*), configurado em graus Celsius, com escala de 0 a 50 °C e faixas de cor alinhadas ao limiar de alerta: verde até 30 °C, amarelo de 30 a 38 °C e vermelho a partir de 38 °C.
3. **Verificar Alertas:** gera a mensagem `ALERTA: BPM simulado elevado` para BPM válido acima de 120; e `ALERTA: Temperatura ambiental elevada` para temperatura acima de 38 °C. Caso nenhuma condição seja atendida, o texto indica `Monitoramento sem alertas ativos`.

Esses valores de corte são **regras demonstrativas**, escolhidas para o exercício acadêmico, e não critérios de diagnóstico. O status textual descreve a avaliação da mensagem recebida naquele instante; na implementação atual, a ausência de BPM válido não é diferenciada visualmente de uma medição válida sem alerta. Essa limitação deve ser corrigida antes de qualquer uso mais crítico.

O gráfico exibe a evolução dos valores válidos recebidos, enquanto o gauge mostra a temperatura da última amostra processada. O fluxo exportado em JSON permite importar a configuração em outra instância compatível de Node-RED, desde que os pacotes necessários estejam instalados.

6. Testes executados e resultados observados

Foram realizados testes funcionais na simulação Wokwi e no dashboard local do Node-RED:

Teste	Procedimento	Evidência observada
Compilação	Executar a versão MQTT do `sketch.ino` no Wokwi	Simulação iniciada e Monitor Serial exibindo leituras
Coleta offline	Manter a transmissão desabilitada durante diversas leituras	Registros adicionados à fila (`[EDGE]`)
Reconexão	Enviar `o` no Monitor Serial	Mensagens de conexão Wi-Fi/MQTT e publicações aceitas para as leituras pendentes
Integração MQTT	Manter Wokwi e Node-RED em execução	Dashboard exibiu dados oriundos da simulação
Gráfico	Produzir pulsos manuais válidos	Curva de BPM exibida, com valores simulados entre aproximadamente 80 e 200 BPM na captura 01
Temperatura	Manter DHT22 com temperatura ambiente de 24 °C	Gauge exibiu 24 °C
Alerta de BPM	Simular pulsos que ultrapassam o limiar de 120 BPM	Texto `ALERTA: BPM simulado elevado` apareceu
Recuperação do alerta de BPM	Parar de acionar o botão e aguardar a expiração da leitura	Texto de alerta desapareceu
Alerta de temperatura	Ajustar o DHT22 simulado para 39 °C	Texto `ALERTA: Temperatura ambiental elevada` apareceu (captura 01, com 39,4 °C)

No gráfico, várias leituras reprocessadas após a reconexão foram apresentadas com instantes de chegada próximos, em vez dos instantes originais de coleta. Uma evolução futura seria usar os identificadores e timestamps de origem para posicionar os registros no tempo e distinguir dados atrasados de dados recebidos em tempo real.

Evidências. As capturas estão em `evidencias/` e são apresentadas na seção seguinte. Elas documentam a integração de ponta a ponta (a mesma leitura #15 aparece no Monitor Serial do Wokwi e no debug do Node-RED) e o alerta de temperatura. Os testes de alerta de BPM e de recuperação do alerta foram observados na simulação, mas não possuem captura própria.

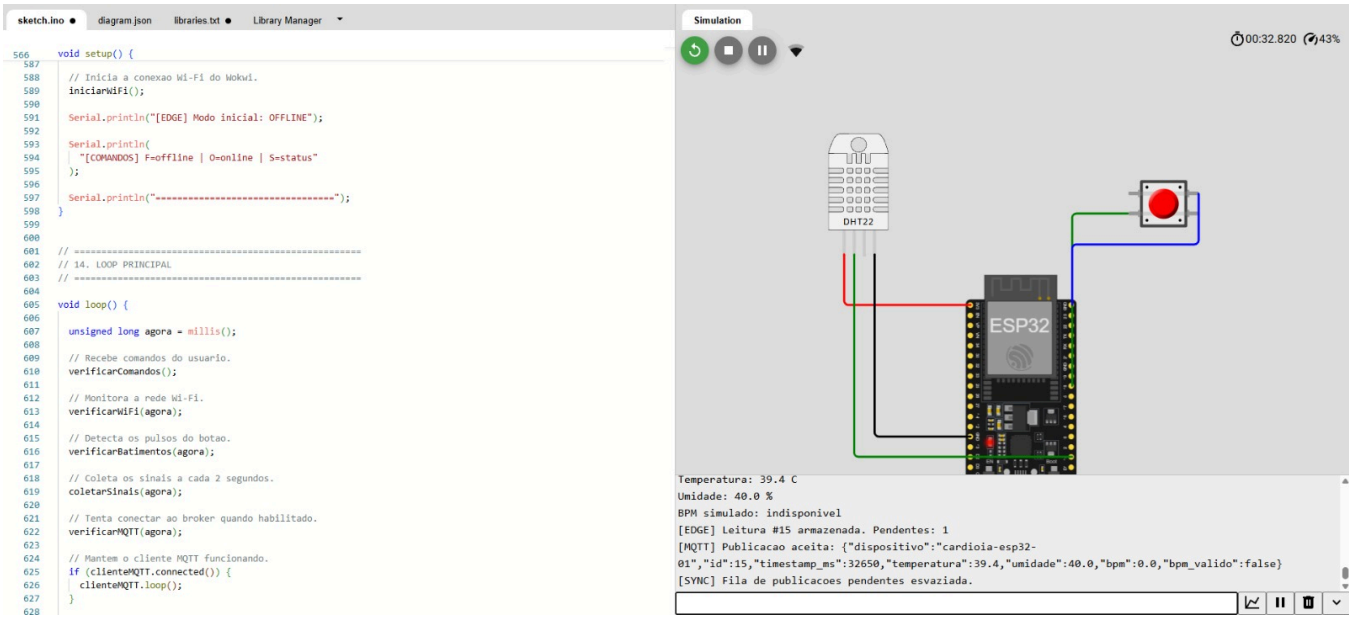
6.1 Capturas de tela

As capturas foram feitas em 08 e 09/10/2026, antes da refatoração final do `sketch.ino`; por isso o texto exato das mensagens do Monitor Serial pode diferir do firmware atual, embora o fluxo demonstrado (coleta, fila, publicação e dashboard) seja o mesmo.

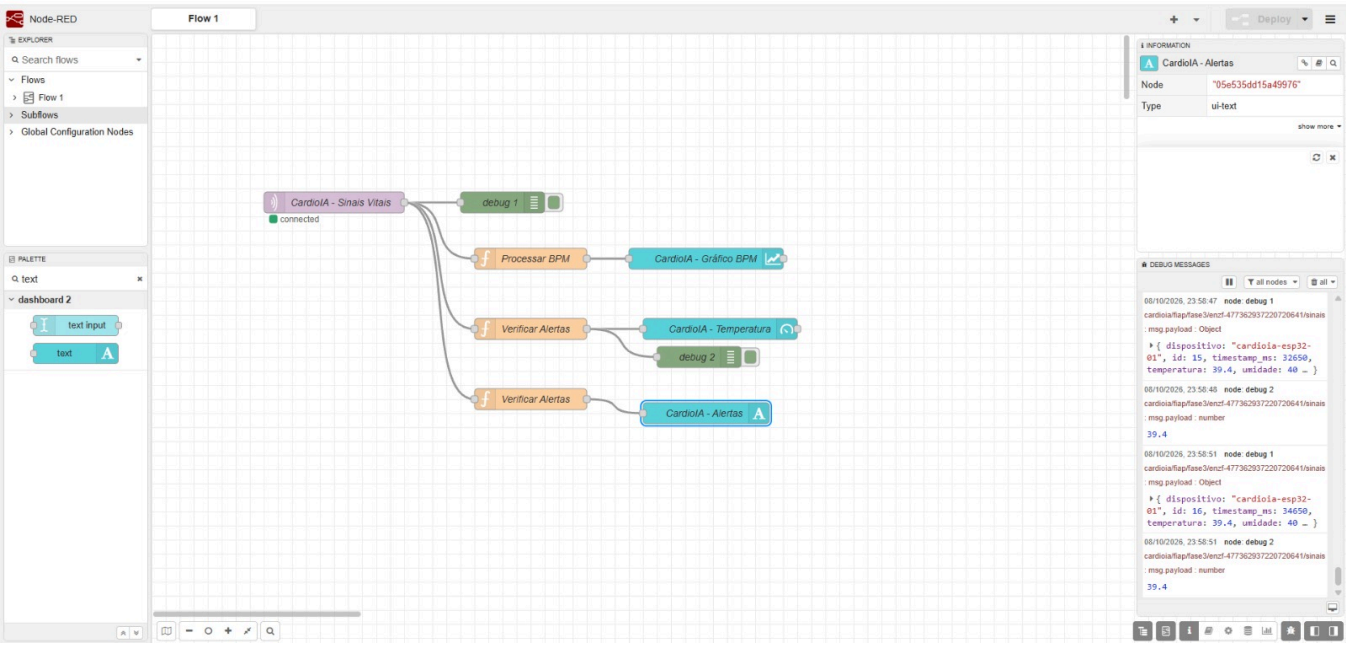
Captura 01 — Dashboard Node-RED com alerta de temperatura (DHT22 em 39,4 °C):



Captura 02 — Wokwi: circuito, Monitor Serial e publicação MQTT da leitura #15:



Captura 03 — Editor do Node-RED: fluxo conectado ao broker e mensagens recebidas (debug):



7. Limitações, segurança e possíveis melhorias

O broker empregado é **público e não criptografado**, adequado exclusivamente a testes com dados fictícios. Não se deve enviar nomes, informações identificáveis, dados reais de saúde, chaves de acesso ou segredos por esse canal. Em ambiente real seria necessário utilizar TLS, autenticação, controle de acesso por tópicos e políticas de privacidade e retenção.

O dispositivo é uma simulação: não existe sensor cardíaco real, avaliação clínica, calibração certificada ou validação de precisão. Além disso, o buffer em RAM não garante persistência após reinicialização, o MQTT em QoS 0 não garante entrega, e `millis()` não corresponde a um relógio sincronizado.

Os limiares de alerta são avaliados somente no Node-RED (Fog): com o dispositivo offline não há alerta algum. Isso é aceitável para a demonstração, mas um monitor real precisaria de um alerta local no próprio ESP32.

Como próximos avanços são recomendados: armazenamento persistente quando aplicável, timestamps sincronizados (por exemplo, NTP), monitoramento do estado *sem dados válidos*, alerta local no dispositivo (LED ou buzzer), tratamento de falhas com confirmações fim a fim, migração para um broker com TLS e autenticação (como o HiveMQ Cloud), controle de acesso e testes automatizados para os limiares de alerta.

8. Conclusão

A Parte 2 integrou o protótipo Edge ao broker MQTT e ao Node-RED, permitindo observar leituras de temperatura e BPM simulado e disparar alertas automáticos por regras definidas. Os testes mostraram a continuidade da coleta durante a indisponibilidade da transmissão, a tentativa de sincronização após a reconexão e a atualização do dashboard com dados recebidos pelo consumidor MQTT.

O resultado atende ao propósito **didático** de demonstrar aquisição, transporte, processamento e visualização de telemetria IoT, deixando explícitas as limitações de segurança, durabilidade e confiabilidade que precisariam ser resolvidas para um sistema de monitoramento de saúde real.

9. Materiais de reprodução

- Simulação Wokwi: https://wokwi.com/projects/477362937220720641
- Firmware: `fase3/esp32/sketch.ino`
- Bibliotecas: `fase3/esp32/libraries.txt`
- Fluxo Node-RED: `fase3/node_red/cardioia-fase3-flow.json`
- Dashboard local (com Node-RED em execução): `http://localhost:1880/dashboard`
- Relatório da Parte 1: `fase3/docs/relatorio_edge.md`
- Evidências (capturas de tela): `fase3/docs/evidencias/`
- Repositório: https://github.com/Grupo-S-faculdade-FIAP/cardioIA